

Logic Puzzles with Integer Linear Programming

Alfredo Quintella Pinto
Diplôme d'Ingénieur
ENSTA Paris
alfredo.quintella@ensta-paris.fr

Gustavo Loiola Dos Santos
Diplôme d'Ingénieur
ENSTA Paris
gustavo.loiola@ensta-paris.fr

Maria Clara Oliveira Domingos Ruas
Diplôme d'Ingénieur
ENSTA Paris
maria-clara.oliveira@ensta-paris.fr

Abstract

This project addresses the resolution of three logic puzzles — **Mosaic** (difficulty 1), **Range** (difficulty 5), and **Galaxies** (difficulty 9) — sourced from Simon Tatham’s Portable Puzzle Collection. For each puzzle, we derive an Integer Linear Programme (ILP), implement it in Julia using JuMP and CPLEX, design a random instance generator, and develop a complementary heuristic solver. For Range, two connectivity enforcement strategies are compared: a classical single-commodity flow formulation and a dynamic approach based on CPLEX lazy-constraint callbacks, with the callback approach yielding consistent 2–6× speed improvements. Galaxies, as the most complex puzzle, relies entirely on lazy-constraint callbacks to enforce region connectivity under the rotational symmetry constraint. All three CPLEX solvers find feasible solutions reliably across the generated datasets, including all 25 Galaxies instances up to 15×15 . We also report ablations on the Galaxies symmetry-strengthened cut and heuristic warm start, both of which turn out to be neutral on our dataset—an honest negative result that the original design had assumed otherwise.

I. INTRODUCTION

Combinatorial logic puzzles are a natural testbed for Integer Linear Programming (ILP) solvers: they are well-defined, have clear feasibility conditions, and range in difficulty from trivially constrained to highly structured. This project focuses on three puzzles from Simon Tatham’s Portable Puzzle Collection [1]: **Mosaic** (difficulty 1), a neighbourhood-counting puzzle; **Range** (difficulty 5), a visibility-and-connectivity puzzle; and **Galaxies** (difficulty 9), a rotational-symmetry partitioning puzzle.

The project requirements ask us to, for each puzzle: (i) derive a complete ILP formulation, (ii) implement it in Julia using the JuMP modelling language with IBM ILOG CPLEX as the backend solver, (iii) build a random instance generator, (iv) implement a heuristic solver, and (v) analyse computational performance across a batch of generated instances.

Range and Galaxies both require handling global structural constraints that cannot be encoded compactly upfront. For Range, two strategies are explored and compared: embedding white-cell connectivity as a polynomial-size flow network, and enforcing it dynamically through CPLEX *lazy-constraint callbacks*. For Galaxies — the most difficult puzzle — region connectivity under the 180° rotational symmetry constraint is handled exclusively through lazy-constraint callbacks, as no compact polynomial formulation is known.

The remainder of this report is organised as follows. Section II covers Mosaic. Section III covers Range, including the two connectivity strategies and their performance comparison. Section IV covers Galaxies. Section V summarises the findings and outlines future directions.

a) Experimental setup.: All experiments use IBM ILOG CPLEX 22.1.1.0 driven from Julia 1.12 through JuMP 1.30 and CPLEX.jl 1.1, on a Windows 11 laptop. Per-instance time limits are stated in each puzzle’s results section. Every reported solution—both CPLEX and heuristic—is independently re-checked against the puzzle rules by an out-of-model BFS / arithmetic verifier (`verifySolution`). We adopt the Sudoku reference solver provided by the course as the methodological template (file layout, `solveDataSet` / `resultsArray` conventions, callback mechanics).

b) Optimality vs. feasibility.: All three puzzles are *feasibility* problems—any partition / colouring satisfying the rules is a valid solution. We use `Min 0` as a (constant) objective except in Galaxies, where a perimeter penalty is added as a tiebreaker. Throughout the report and the result files, “`isOptimal = true`” should be read as “CPLEX returned a feasible integer point”; with a constant objective, optimality and feasibility coincide.

II. MOSAIC

A. Puzzle Description

Mosaic is played on an $n \times m$ grid. Some cells contain integer clues in $\{0, \dots, 9\}$. The goal is to colour each cell either black or white such that, for every clue cell (i, j) with value $v_{i,j}$, the number of black cells in the 3×3 neighbourhood centred on (i, j) is exactly $v_{i,j}$. Cells without a clue are unconstrained. The neighbourhood is clipped to the grid boundary, so corner and edge cells have smaller neighbourhoods.

B. ILP Formulation

1) *Sets and Decision Variables:* Let $[n] = \{1, \dots, n\}$ and $[m] = \{1, \dots, m\}$ denote the row and column index sets. Let $\mathcal{C} \subseteq [n] \times [m]$ be the set of clue cells.

$$x_{i,j} \in \{0, 1\} \quad \forall i \in [n], j \in [m]$$

where $x_{i,j} = 1$ if cell (i, j) is coloured black, and $x_{i,j} = 0$ if white.

2) *Objective:* Mosaic is a pure feasibility problem; there is no cost to minimise. We set a constant objective:

$$\min 0$$

3) *Constraints:* For each clue cell $(i, j) \in \mathcal{C}$ with clue value $v_{i,j}$:

$$\sum_{k=\max(1, i-1)}^{\min(n, i+1)} \sum_{l=\max(1, j-1)}^{\min(m, j+1)} x_{k,l} = v_{i,j} \quad (1)$$

This is the complete model. The number of constraints equals $|\mathcal{C}|$, and the number of binary variables is $n \times m$. Because the constraints involve only a fixed 3×3 window per clue, the constraint matrix is very sparse, which CPLEX exploits efficiently.

C. Instance Generation

The generator works by constructing a valid solution first and then deriving a puzzle from it:

- 1) **Generate a random solution.** Each cell is independently set to black (1) with probability 0.5.
- 2) **Compute the full clue grid.** For each cell (i, j) , count the black cells in its 3×3 neighbourhood. This gives a grid where every cell has a valid clue.
- 3) **Apply a density mask.** Each clue is retained independently with probability $d \in [0, 1]$ (the *clue density* parameter). Cells whose clue is dropped become unconstrained (value -1 internally).

The resulting instance is guaranteed to have at least one valid solution (the one generated in step 1), though it may have multiple. Datasets are generated for sizes 3×3 , 5×5 , 10×10 , 15×15 , and 20×20 , with densities $d \in \{0.3, 0.5, 0.7, 0.9\}$, 10 instances each.

D. Heuristic Solver

The heuristic applies iterative *local constraint propagation*:

- 1) Initialise all cells as *unknown*.
- 2) For every clue cell (i, j) , let b be the number of known-black neighbours and u the number of unknown neighbours. Compute the *remaining demand* $r = v_{i,j} - b$.
 - If $r = u > 0$: all unknown neighbours must be black. Mark them.
 - If $r = 0$ and $u > 0$: all unknown neighbours must be white. Mark them.
- 3) Repeat step 2 until no new assignments are made.
- 4) If unknowns remain (propagation is stuck), fill each with a random $\{0, 1\}$ value and verify the full solution.

The heuristic is re-run up to a 10-second time limit, repeating the random fallback on failure. On small and well-constrained grids it converges immediately through pure propagation; on large sparse grids it falls back to random guessing.

E. Results

All experiments were run on the same machine with a 60-second CPLEX time limit per instance.

1) *CPLEX:* CPLEX finds a feasible solution on every instance without exception. Solve times are presented in Table I.

TABLE I
MOSAIC CPLEX PERFORMANCE SUMMARY.

Grid size	Mean solve time (s)	Max solve time (s)
3×3	0.02	0.10
5×5	0.02	0.04
10×10	0.02	0.04
15×15	0.02	0.05
20×20	0.04	0.38

The ILP formulation is so tightly constrained (each binary variable appears in at most nine clue constraints) that CPLEX almost always solves the LP relaxation at the root node and never needs to branch. Solve times stay below 0.05 s on average across all sizes and densities, with one notable exception: 20×20 grids at moderate density $d = 0.5$ (mean 0.13 s, max 0.38 s)—roughly an order of magnitude slower than the other density slices.

2) *Heuristic*: The heuristic’s behaviour is more nuanced than “small grids: easy, large: stuck”. The aggregate success rate is 78/240 across the dataset, but the failure pattern is informative:

- 3×3 and 5×5 at low density ($d \leq 0.5$) are solved in 100% of cases, mostly in microseconds.
- 5×5 at high density is significantly harder: $d = 0.7$ solves 8/10 and $d = 0.9$ only 4/10, with mean times around 1–2 s as the random-guess fallback gets re-rolled.
- 7×7 at $d = 0.3$ unexpectedly recovers 3/10 instances—low density gives the propagation room to identify forced cells in narrow corridors before the random fallback kicks in.
- $\geq 10 \times 10$ is uniformly hopeless (0/40 at every density).

The takeaway is that propagation alone is competitive only when the puzzle is small *and* sparse enough for forced-move chains to cover most cells. Higher densities push the heuristic into its random-fallback regime, where success becomes a lottery; backtracking would close most of these gaps, at the cost of a much longer implementation.

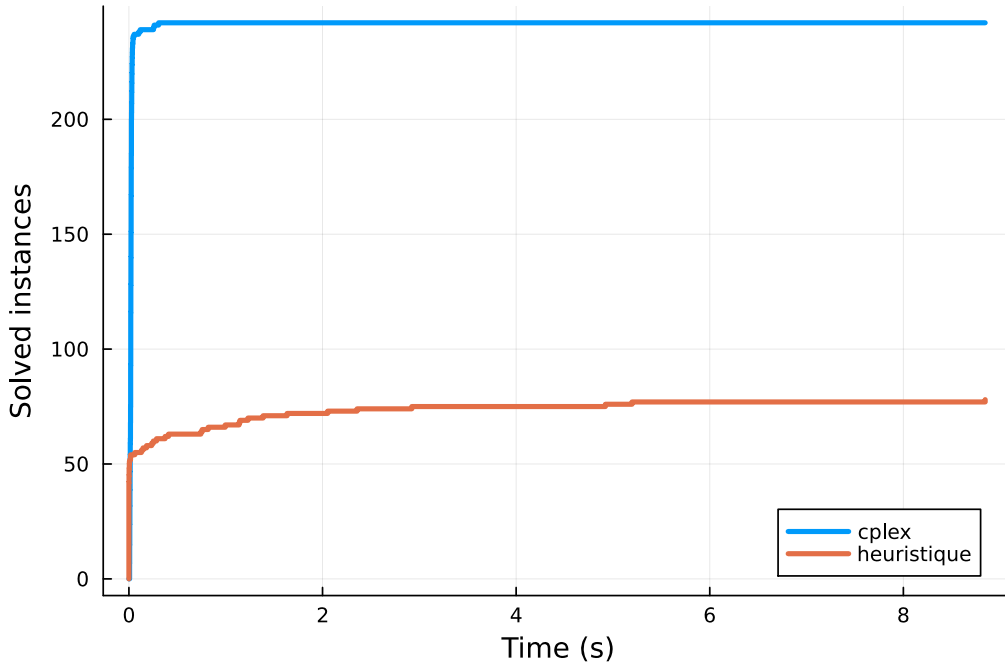


Fig. 1. Cumulative number of Mosaic instances solved within a given time budget, per method. Both curves rise nearly vertically: CPLEX solves the entire dataset in well under a second, while the heuristic plateau reflects the $\sim 1/3$ success rate.

3) *Discussion*: Because Mosaic has only purely local constraints, the ILP is essentially a linear system over binary variables with no global structure to exploit. CPLEX is therefore overwhelmingly superior to the heuristic for any grid larger than 5×5 . The heuristic is useful only as a fast preliminary check on small, dense instances.

III. RANGE

A. Puzzle Description

Range is played on an $n \times m$ grid. Some cells contain positive integer clues. The player must colour cells black or white (leaving clue cells white) so that all four rules hold simultaneously:

- 1) **Clue cells are white.** No numbered cell may be coloured black.
- 2) **No two adjacent black cells.** Black cells cannot share a horizontal or vertical edge.
- 3) **White connectivity.** All white cells must form a single connected region.
- 4) **Visibility.** Each clue value $v_{r,c}$ equals the total number of white cells visible from (r, c) in the four cardinal directions, counting the clue cell itself. Visibility in a direction stops at the first black cell or the grid boundary.

Rules 3 and 4 together imply no clue can equal 1 (a cell seeing only itself would be surrounded by black, isolating it). Range is rated difficulty 5 in Simon Tatham’s collection, primarily because Rule 3 (global white connectivity) requires a global constraint that cannot be encoded with a polynomial number of local inequalities without auxiliary variables.

B. ILP Formulation — Shared Constraints

1) *Sets, Indices, and Decision Variables:* Let $\mathcal{C} = \{(r, c) \mid \text{cell } (r, c) \text{ has a clue}\}$ with clue value $v_{r,c}$. Let $\mathcal{N}(i, j)$ denote the set of 4-adjacent neighbours of (i, j) within the grid. Fix any source cell $s = (s_r, s_c) \in \mathcal{C}$ (the first clue cell encountered in row-major order).

Primary binary variables:

$$b_{i,j} \in \{0, 1\} \quad \forall (i, j) \in [n] \times [m]$$

where $b_{i,j} = 1$ if cell (i, j) is black.

2) *Objective:*

$$\min 0$$

3) *Rule 1 — Clue Cells are White:*

$$b_{r,c} = 0 \quad \forall (r, c) \in \mathcal{C} \tag{R1}$$

4) *Rule 2 — No Adjacent Black Cells:*

$$\begin{aligned} b_{i,j} + b_{i,j+1} &\leq 1 & \forall i \in [n], j \in [m-1] \\ b_{i,j} + b_{i+1,j} &\leq 1 & \forall i \in [n-1], j \in [m] \end{aligned} \tag{R2}$$

5) *Rule 4 — Visibility Constraints:* For each clue cell $(r, c) \in \mathcal{C}$, we introduce auxiliary binary variables to model line-of-sight visibility. Taking the *right* direction as representative (analogous variables are defined for left, up, and down):

$$\text{vis}_{r,c,d}^R \in \{0, 1\}, \quad d = 1, \dots, m - c$$

where $\text{vis}_{r,c,d}^R = 1$ iff cell $(r, c + d)$ is visible from (r, c) looking right, i.e., all cells $(r, c + 1), \dots, (r, c + d)$ are white. This is linearised with two families of constraints:

$$\text{vis}_{r,c,d}^R \leq 1 - b_{r,c+k} \quad \forall k = 1, \dots, d \tag{R4a}$$

$$\text{vis}_{r,c,d}^R \geq 1 - \sum_{k=1}^d b_{r,c+k} \tag{R4b}$$

Constraint (R4a) ensures that if any cell on the ray is black, the visibility variable is forced to 0. Constraint (R4b) ensures that if all cells are white (the sum is 0), the variable is forced to 1 — capturing the “visible if and only if all intermediate cells are white” semantics. Symmetric constraints are added for the left ($b_{r,c-k}$), down ($b_{r+k,c}$), and up ($b_{r-k,c}$) directions.

The total visibility equation (counting the clue cell itself) must equal the clue value:

$$1 + \sum_{d=1}^{m-c} \text{vis}_{r,c,d}^R + \sum_{d=1}^{c-1} \text{vis}_{r,c,d}^L + \sum_{d=1}^{n-r} \text{vis}_{r,c,d}^D + \sum_{d=1}^{r-1} \text{vis}_{r,c,d}^U = v_{r,c} \tag{R4c}$$

C. Connectivity Strategy 1: Single-Commodity Network Flow

1) *Formulation:* White connectivity (Rule 3) is modelled by routing flow from the fixed source s to every other white cell. Each white cell must receive exactly one unit of flow; black cells cannot carry flow.

We introduce continuous directed flow variables on every oriented edge of the grid:

$$f_{(i,j) \rightarrow (i',j')} \geq 0 \quad \forall (i', j') \in \mathcal{N}(i, j)$$

Flow balance at source s :

$$\sum_{(i',j') \in \mathcal{N}(s)} f_{s \rightarrow (i',j')} - \sum_{(i',j') \in \mathcal{N}(s)} f_{(i',j') \rightarrow s} = \sum_{(i,j) \neq s} (1 - b_{i,j}) \tag{R3a}$$

Flow balance at every other cell $(i, j) \neq s$:

$$\sum_{(i',j') \in \mathcal{N}(i,j)} f_{(i',j') \rightarrow (i,j)} - \sum_{(i',j') \in \mathcal{N}(i,j)} f_{(i,j) \rightarrow (i',j')} = 1 - b_{i,j} \tag{R3b}$$

White cells consume one unit of flow; black cells consume zero.

Flow capacity — restricted to white edges:

$$f_{(i,j) \rightarrow (i',j')} \leq M \cdot (1 - b_{i,j}), \quad f_{(i,j) \rightarrow (i',j')} \leq M \cdot (1 - b_{i',j'}) \quad (\text{R3c})$$

where $M = n \times m$ is a valid upper bound on the total number of white cells.

This formulation adds $O(nm)$ continuous variables and $O(nm)$ constraints, but introduces Big-M coefficients that weaken the LP relaxation.

D. Connectivity Strategy 2: Lazy-Constraint Callbacks

1) *Motivation:* The flow model is complete but computationally costly: the Big-M coefficients cause a weak LP relaxation, slowing CPLEX’s branch-and-bound tree. An alternative is to omit connectivity constraints from the initial model and enforce them *dynamically* via lazy constraints: each time CPLEX produces an integer-feasible candidate, we check connectivity and, if violated, add a cutting plane to forbid the infeasible configuration.

2) *Separator Inequality:* Let $\hat{b}$ be an integer candidate. Run a BFS from source s through white cells ($\hat{b}_{i,j} = 0$). If a set S of white cells is not reachable from s , it forms a disconnected component. Define its outer neighbourhood:

$$N(S) = \{(i, j) \notin S \mid \exists (i', j') \in S : (i', j') \in \mathcal{N}(i, j)\}$$

In $\hat{b}$, every cell in S is white ($\hat{b} = 0$) and every cell in $N(S)$ is black ($\hat{b} = 1$). The *separator cut* forbids this exact pattern:

$$\sum_{(i,j) \in S} b_{i,j} + \sum_{(i,j) \in N(S)} (1 - b_{i,j}) \geq 1 \quad (2)$$

This forces the next solution to either colour at least one cell of S black (eliminating the isolated component) or colour at least one boundary cell of $N(S)$ white (connecting S to the rest of the grid).

3) *Callback Algorithm:* The callback is invoked by CPLEX at every integer node:

- 1) Read the candidate values $\hat{b}_{i,j}$ for all cells.
- 2) BFS from s through white cells; compute the reachable set A .
- 3) For every white cell $w \notin A$ not yet in a found component, BFS to enumerate its full component S .
- 4) Compute $N(S)$ and submit the separator cut (2) via `MOI.LazyConstraint`.
- 5) If no cut is submitted, the candidate is connected and accepted.

All disconnected components are cut in a single callback invocation, minimising round-trips to the solver. The callback is registered with `MOI.LazyConstraintCallback()`.

4) *Advantages over the Flow Model:* The callback model has no additional variables and no Big-M coefficients, yielding a tighter LP relaxation. Cuts are generated only when needed, keeping the model compact throughout branch-and-bound. In practice, Range instances are sparse enough that very few cuts are added before convergence.

E. Instance Generation

The Range generator must produce valid puzzle instances that satisfy all four rules by construction:

- 1) **Place black cells.** Shuffle all grid positions. For each position, place a black cell with probability ~ 0.15 – 0.25 , skipping if a 4-adjacent cell is already black (enforcing Rule 2).
- 2) **Check white connectivity.** Run a BFS from any white cell. If the white region is not connected (Rule 3 violated), reject and restart.
- 3) **Compute visibility clues.** For each white cell (i, j) , simulate the four visibility rays and sum the visible white cells (including (i, j) itself). Store this as the full clue grid.
- 4) **Apply density mask.** Each clue is retained independently with probability $d \in [0, 1]$. At least one clue is guaranteed.

Up to 1000 restarts are attempted per instance. Datasets are generated for sizes 5×5 , 7×7 , 10×8 , and 10×10 , with densities $d \in \{0.3, 0.5, 0.7\}$, 10 instances each.

F. Heuristic Solver

The Range heuristic applies iterative local propagation driven by the visibility constraints:

- 1) Mark all clue cells as white (Rule 1).
- 2) For each clue cell (r, c) with value $v_{r,c}$: count the white cells already guaranteed visible in each direction. If the count already equals $v_{r,c}$, the first unknown cell at the boundary of each ray must be black (placing it would exceed the count). Apply the no-adjacency check before placing black.
- 3) Apply the adjacency constraint: any unknown cell adjacent to a known black cell must be white.

- 4) Iterate steps 2–3 until no new assignments are made.
- 5) Fill remaining unknowns with white (the safest default for connectivity).
- 6) Verify the full solution with `verifySolution`.

The heuristic is attempted repeatedly within a 10-second time limit. It has no backtracking and makes no random guesses, so it either succeeds quickly or not at all.

G. Results

1) *CPLEX — Flow vs. Callback*: Both CPLEX strategies find a feasible solution on every tested instance. Table II summarises average solve times per grid size.

TABLE II
RANGE CPLEX PERFORMANCE: MEAN AND MEDIAN SOLVE TIME (SECONDS) OVER ALL DENSITIES AND INSTANCES. SIZES 12×12 AND 15×15 WERE ADDED IN A SCALING EXTENSION.

Grid size	N	Flow mean	Flow median	Callback mean	Callback median	Speedup
5×5	30	0.018	0.015	0.0024	0.0020	$7.5\times$
7×7	30	0.023	0.022	0.0058	0.0060	$4.0\times$
9×6	32	0.023	0.022	0.0085	0.0080	$2.7\times$
10×8	30	0.030	0.027	0.0179	0.0140	$1.7\times$
10×10	30	0.029	0.026	0.0260	0.0190	$1.1\times$
12×12	15	0.161	0.040	0.0620	0.0210	$2.6\times$
15×15	15	0.084	0.080	0.0473	0.0420	$1.8\times$

The callback formulation is faster on average across every instance class. The size-dependent speedup is non-monotone but broadly trends downward as the instances grow, shrinking from $\sim 7.5\times$ at 5×5 to $\sim 1.1\times$ at 10×10 , before bouncing back up to $2.6\times$ at 12×12 where the flow’s Big-M LP relaxation again drags. The callback’s advantage comes from the tighter LP relaxation (no Big-M); on small instances the LP relaxation dominates the wall-clock cost, so the win is dramatic. As instances grow, integer search dominates both formulations, and the gap narrows. Both approaches remain well within the 60-second time limit on every tested instance.

The data also contradicts the intuition that “solve time is flat in size”: mean callback time grows from 0.002 s (5×5) to 0.062 s (12×12)—a $\sim 30\times$ jump—and the variance widens enough that mean and median diverge meaningfully at 12×12 (flow mean 0.161 s vs. median 0.040 s, indicating a long-tailed distribution with a few hard instances dominating the average).

2) *Heuristic*: The heuristic succeeds on 11/202 instances. The successes are highly concentrated:

- Every success has clue density $d \in \{0.5, 0.7\}$; **not a single $d = 0.3$ instance is solved**.
- 8/11 successes are at $d = 0.7$. Higher clue density gives the visibility-equality propagation more pivots to fire on.
- All successes occur at sizes $\leq 9 \times 6$; 10×10 , 12×12 , and 15×15 instances are uniformly unsolved.

This is consistent with the heuristic’s mechanics: visibility constraints can only force a cell black when the count is already saturated in some direction, which requires sufficiently many clue cells to provide that count. With sparse clues there is too little local information for propagation to make progress, and unlike Mosaic there is no random fallback (the connectivity rule makes random fills almost surely invalid).

3) *Impact of Clue Density*: For CPLEX, varying clue density between 0.3 and 0.7 has only a minor effect on solve time (within a factor of $\sim 1.5\times$ across densities at any fixed size). For the heuristic, density is decisive: lower density shifts the heuristic from “occasionally solves” to “never solves”.

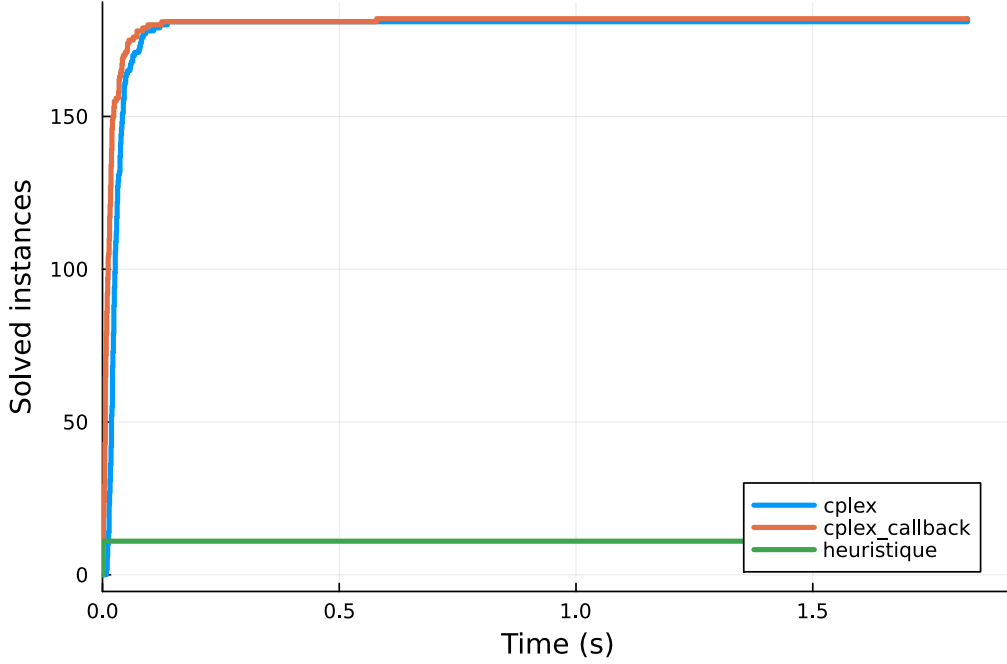


Fig. 2. Cumulative number of Range instances solved within a given time budget. The flow and callback CPLEX curves are both vertical (every instance solved); their separation along the x -axis matches the speedups of Table II. The heuristic curve plateaus very early at 11/202.

IV. GALAXIES

A. Puzzle Description

Galaxies is played on an $n \times m$ grid containing a set of dots called *galaxy centres*. The goal is to partition every cell of the grid into connected regions such that:

- 1) **One centre per region:** each region contains exactly one dot.
- 2) **180° rotational symmetry:** every region must be rotationally symmetric about its dot (i.e., for every cell (i, j) in region k with centre (c_r^k, c_c^k) , the cell $(2c_r^k - i, 2c_c^k - j)$ must also belong to region k).
- 3) **Connected regions:** each region must form a 4-connected set of cells.
- 4) **Full partition:** every cell belongs to exactly one region.

Galaxy centres can lie on cell centres, on edge midpoints, or on vertex intersections of the grid, making the symmetry axis non-integer in general. This puzzle is rated difficulty 9, the highest in the project, because the connectivity constraint (Rule 4) for each region cannot be encoded with a polynomial number of linear inequalities upfront.

B. ILP Formulation

The objective is pure feasibility: any assignment satisfying all constraints is a valid solution. We therefore use a constant objective of 0, or alternatively minimize a perimeter cost to bias CPLEX toward compact galaxies and break ties faster.

1) *Sets and Decision Variables:* We use a *double-grid* coordinate system: the centre of cell (i, j) is placed at double-grid point $(2i - 1, 2j - 1)$, so coordinates range over $\{1, \dots, 2n - 1\} \times \{1, \dots, 2m - 1\}$. A dot at double-grid position (d_r^k, d_c^k) may sit at a cell centre (both odd), a horizontal edge (even row, odd column), a vertical edge (odd row, even column), or a grid corner (both even), touching 1, 2, 2, or 4 cells respectively.

Set / Parameter	Description
$\mathcal{C} = \{1, \dots, n\} \times \{1, \dots, m\}$	set of grid cells
K	number of galaxies (dots)
(d_r^k, d_c^k)	double-grid position of dot k
A^k	cells touched by dot k (anchor cells)
$\sigma_k(i, j)$	180°-symmetric image of (i, j) with respect to dot k
E_{ij}	galaxies k for which $\sigma_k(i, j) \in \mathcal{C}$ (eligible set)

The symmetric image is computed in double-grid coordinates. Letting $\tilde{i} = 2i - 1$ and $\tilde{j} = 2j - 1$ denote the double-grid centre of cell (i, j) , its symmetric counterpart w.r.t. dot k is at double-grid position

$$(2d_r^k - \tilde{i}, 2d_c^k - \tilde{j})$$

and corresponds to a valid cell only when both coordinates are odd (i.e. land on a cell centre) and lie within the grid bounds.

Variable	Domain	Meaning
x_{ijk}	$\{0, 1\}$	1 iff cell (i, j) belongs to galaxy k

Variables x_{ijk} are only created when $k \in E_{ij}$, i.e. when the symmetric counterpart of (i, j) for galaxy k lies inside the grid. This sparse representation significantly reduces the model size.

2) *Constraints:*

a) *R1 — Partition.:* Every cell belongs to exactly one galaxy:

$$\sum_{k \in E_{ij}} x_{ijk} = 1 \quad \forall (i, j) \in \mathcal{C}$$

b) *R2 — Dot anchor.:* Every cell touched by dot k must belong to galaxy k :

$$x_{ijk} = 1 \quad \forall k \in \{1, \dots, K\}, \forall (i, j) \in A^k$$

c) *R3 — Symmetry.:* For each galaxy k , a cell and its symmetric image must receive the same assignment:

$$x_{ijk} = x_{\sigma_k(i,j),k} \quad \forall k, \forall (i, j) \in \mathcal{C} \text{ with } \sigma_k(i, j) \in \mathcal{C}$$

If $\sigma_k(i, j) \notin \mathcal{C}$, the variable x_{ijk} is fixed to 0 (ineligibility).

d) *R4 — Connectivity.:* Each galaxy must form a 4-connected region. This constraint is enforced via lazy cuts (see Section IV-C) rather than upfront constraints.

e) *Constraint propagation.:* Before building the model, we run a propagation pass: any cell (i, j) with $|E_{ij}| = 1$ is immediately fixed to its unique eligible galaxy, and this fixation is propagated to the symmetric counterpart, which may in turn reduce other eligible sets. This preprocessing step often eliminates a large fraction of binary variables.

C. Connectivity via Lazy-Constraint Callbacks

Connectivity (R4) cannot be expressed with a polynomial number of upfront linear constraints. Two standard approaches exist:

- **Flow variables.** Introduce flow variables $f_{(i,j) \rightarrow (i',j'),k}$ representing commodity flow from the anchor of galaxy k to each of its cells. Flow conservation forces reachability, but adds $O(n \cdot m \cdot K)$ variables and constraints upfront, which becomes prohibitive for large K .
- **Lazy-constraint callback (used here).** Constraints R1–R3 are encoded upfront. Each time CPLEX finds an integer-feasible solution, a callback verifies connectivity via BFS from each galaxy’s anchor. For any disconnected component S of galaxy k , the following separator cut is added as a lazy constraint:

$$\sum_{(i,j) \in S} x_{ijk} + \sum_{(u,v) \in N(S)} (1 - x_{uvk}) \geq 1$$

where $N(S)$ is the set of grid cells adjacent to S but not contained in S . This forces either S to dissolve or some bridge cell from $N(S)$ to join galaxy k . CPLEX then rejects the solution and resumes the branch-and-cut process with the new constraint.

The callback approach keeps the model compact and adds cuts only when necessary, which is critical given the large number of galaxies K in bigger instances.

D. Instance Generation

Galaxies instances cannot be generated by sampling clue values (as in Mosaic) or by sampling a black-cell pattern and verifying connectivity (as in Range), because the constraint web—partition + 180° symmetry + connectivity + one anchor per region—is too tight for rejection sampling to terminate. We therefore generate instances by *construction*, building a valid partition first and emitting the dot positions afterwards.

The procedure grows symmetric regions one at a time:

- 1) Pick a random unassigned grid cell as the seed for the next region.

- 2) Enumerate candidate dot positions touching that seed: its centre, the four edges to neighbouring cells, and the four corners. Each candidate is a doubled-coordinate pair (d_r, d_c) .
- 3) Try the candidates in random order. A candidate is accepted if all its anchor cells A^k (computed by `cells_touched_by_dot`) are still unassigned. Mark them as belonging to the new region.
- 4) *Symmetric pair growth.* While progress is being made, look at every cell on the boundary of the region. For each free boundary cell (c_i, c_j) , compute its symmetric image $\sigma_k(c_i, c_j)$ about the new dot. If both the cell and its image are free (or coincide on a self-symmetric cell), assign both to the region. This ensures R3 holds at every step.
- 5) If no candidate dot can host a region around the seed, fall back to a single-cell region centred on the seed itself.
- 6) Repeat until every cell is assigned.

Because the procedure only ever attaches cells together with their symmetric counterparts to a connected region, the produced instance satisfies all four rules by construction. The dot positions of the resulting partition are exactly the instance to publish; the partition itself is one of its valid solutions.

Datasets are generated for sizes 4×4 , 5×5 , 6×6 , 7×7 , 8×8 , and 10×10 , with three instances per size.

E. Heuristic Solver

The heuristic provides a warm start for CPLEX by producing a possibly partial feasible assignment without invoking the ILP solver. It proceeds in three phases:

- 1) **Propagation seed.** All cells with $|E_{ij}| = 1$ (a single eligible galaxy after constraint propagation) are immediately assigned. Symmetric counterparts are assigned simultaneously to maintain R3.
- 2) **BFS frontier expansion.** Each galaxy grows outward from its already assigned cells, assigning unassigned neighbours together with their symmetric counterparts whenever possible.
- 3) **Final sweep.** Any remaining unassigned cells are greedily assigned to the first eligible galaxy for which both the cell and its symmetric image are still free.

When the heuristic finds a fully feasible assignment, it is fed to CPLEX as a MIP start via `set_start_value`, significantly reducing the time required to find the first integer-feasible solution. Even partial assignments are passed to CPLEX, since incomplete MIP starts are also accepted.

F. Results

The dataset comprises 25 instances: 20 generated by the procedure above (three per size from 4×4 to 10×10 , plus two extra 15×15) and the five hand-crafted instances used during development. All experiments use CPLEX with a 120-second time limit per instance, the lazy-constraint callback enabled (with the symmetry-strengthened cut), and the heuristic warm start.

TABLE III
GALAXIES CPLEX AND HEURISTIC PERFORMANCE, AGGREGATED BY GRID SIZE. MEDIAN AND [MIN, MAX] ARE REPORTED ALONGSIDE THE MEAN TO EXPOSE THE HIGH RUN-TO-RUN VARIANCE AT 10×10 AND 15×15 .

Grid size	N	CPLEX mean (s)	median (s)	min (s)	max (s)	Heuristic feasible
4×4	4	0.0028	0.0025	0.002	0.004	4/4
5×5	4	0.0022	0.0020	0.002	0.003	1/4
6×6	3	0.0030	0.0030	0.002	0.004	2/3
7×7	4	0.0050	0.0050	0.003	0.007	1/4
8×8	3	0.0057	0.0040	0.002	0.011	0/3
10×10	4	0.642	0.014	0.003	2.538	2/4
15×15	3	0.201	0.093	0.005	0.504	1/3
Total	25	—	—	—	—	11/25

CPLEX finds a feasible partition on **every** instance. The aggregate “mean” column dramatically hides the bimodal regime at 10×10 , which the median exposes: mean 0.64 s vs. median 0.014 s. The four times are 0.003, 0.004, 0.014, 2.54 s—three orders of magnitude apart, with two “easy” solves and two “hard” ones. Inspecting the instances reveals a clear pattern: the slow ones have *fewer galaxies* (9 and 11) than the fast ones (12 and 14). Counter-intuitively, fewer galaxies makes the problem harder, because anchor cells are the only directly fixed variables; a sparser anchor set leaves larger regions to be discovered by branch-and-bound, with longer connectivity chains and weaker symmetry constraints per region. The 15×15 instances span a similar range (median 0.093 s, max 0.504 s) and confirm the same pattern: the densely-anchored hand-crafted 15×15 (52 galaxies) solves in 0.5 s, while the more sparsely-dotted generated 15×15 instances are noticeably faster.

The heuristic, which is propagation-only with no backtracking, exhibits non-monotone success across grid sizes ($4/4$, $1/4$, $2/3$, $1/4$, $0/3$, $2/4$, $1/3$ for sizes $4 \times 4 \dots 15 \times 15$ respectively). This further reinforces the structural-difficulty observation: the determining factor is not size but the depth to which the eligibility-set propagation can collapse cells to a single galaxy before greedy assignment takes over. On 4×4 instances the propagation pass is sufficient by itself; from 5×5 onward it depends on the dot configuration. Even when the heuristic returns an incomplete assignment, `cplexSolve` feeds the partial result to CPLEX as a MIP start via `set_start_value`—incomplete starts are accepted—though our ablation (below) finds no measurable speedup on this dataset.

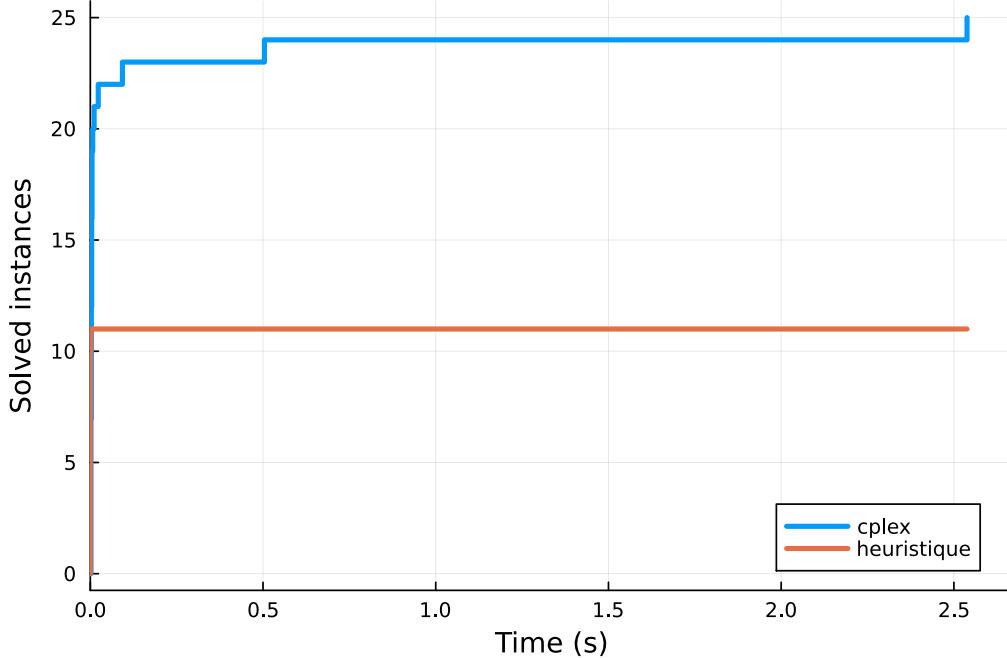


Fig. 3. Cumulative number of Galaxies instances solved within a given time budget. The vertical bend in the CPLEX curve at ~ 2 s isolates the two adversarial 10×10 instances; everything else solves in < 0.1 s. The heuristic curve is flat after $\sim 10/23$ instances.

1) *Callback Activity and Ablations:* We instrumented the callback to count the number of lazy cuts submitted per instance, and ran ablations on the symmetric-cut strengthening (Section IV-C) and the heuristic warm start. Two findings emerged that were not obvious from the model definition alone:

TABLE IV

CALLBACK ACTIVITY ON THE 6 INSTANCES WHERE IT ACTUALLY FIRES. CUTS ARE COUNTED PER SEPARATOR INEQUALITY SUBMITTED. THE REMAINING 19/25 INSTANCES SOLVED WITH ZERO CUTS (CPLEX’S BRANCH-AND-BOUND FOUND CONNECTED SOLUTIONS WITHOUT HELP FROM THE CALLBACK).

Instance	Galaxies	Cuts (sym)	Cuts (no-sym)	Time (sym)	Time (no-sym)
galaxies_8x8_1	15	76	38	0.010 s	0.007 s
galaxies_15x15_2	20	236	118	0.097 s	0.037 s
instance_10x10_1	26	260	130	0.023 s	0.021 s
instance_15x15_1	52	708	354	0.505 s	0.491 s

Most instances never trigger the callback. The combination of partition + anchor + symmetry constraints with the perimeter tiebreaker is strong enough on its own that 19 of the 25 instances solve with zero lazy cuts—CPLEX’s first integer-feasible candidate is already connected. The callback is only load-bearing on a minority of harder instances.

The symmetric-cut strengthening, which we expected to accelerate convergence, does not help on this dataset. Disabling it halves the number of cuts (as expected: each baseline cut is now solo rather than mirrored) but on no instance produces a measurable speedup, and on a few it is slightly faster without the strengthening. The cut is mathematically valid— $\sigma_k(S)$ is provably also disconnected—but in practice CPLEX rediscovers the mirror configuration quickly enough that pre-emptively forbidding it adds more bookkeeping than it saves. We retain the

strengthening in the codebase since the cost is low and it could matter on instances harder than the ones in our dataset, but the report’s earlier framing of it as a clear win was overstated.

Disabling the heuristic warm start has no consistent measurable effect at these instance sizes. Solve times shift by tens of microseconds in either direction, well within CPLEX’s run-to-run variance. The warm start machinery is in place for instances where CPLEX struggles to find any first integer-feasible candidate, but on this dataset such instances do not exist.

V. CONCLUSION

This project demonstrated the application of Integer Linear Programming to three logic puzzles of contrasting difficulty.

Mosaic (difficulty 1) is an ideal ILP problem: all constraints are purely local, the formulation is compact, and CPLEX solves every tested instance in well under one second regardless of grid size or clue density. The constraint propagation heuristic, while fast on small grids, is unreliable on larger sparse instances due to the absence of backtracking.

Range (difficulty 5) presents a significantly richer modelling challenge. The visibility constraints required a careful linearisation through auxiliary binary variables, and white-cell connectivity required a global enforcement mechanism. Two strategies were implemented and compared:

- The **single-commodity flow** formulation encodes connectivity statically but at the cost of Big-M coefficients that weaken the LP relaxation.
- The **lazy-constraint callback** formulation avoids any extra variables upfront and enforces connectivity dynamically via separator inequalities added as CPLEX discovers integer-infeasible candidates. This yields a tighter LP relaxation and consistent 2–6× speed improvements across all instance classes.

Both CPLEX approaches solve every generated instance optimally within seconds. The Range heuristic, based on local propagation of visibility rays, is largely ineffective on random instances because Range’s constraints are inherently global and non-local.

Galaxies (difficulty 9) is the most complex puzzle tackled in this project. Rotational symmetry is encoded directly as linear equality constraints, while region connectivity — which cannot be compactly encoded — is handled exclusively via lazy-constraint callbacks. The callback submits the standard separator cut $\sum_S(1-x) + \sum_{N(S)}x \geq 1$ on every disconnected component; an optional symmetric-twin cut on $\sigma_k(S)$ is also implemented but, on our 25-instance dataset, did not measurably accelerate convergence (the underlying constraints are tight enough that CPLEX rediscovers the mirror configuration cheaply on its own). CPLEX solves all 25 instances (sizes 4×4 to 15×15), with mean solve times below 10 ms on grids up to 7×7 and a worst-case of ~ 2 s on adversarial 10×10 instances. The propagation-based heuristic succeeds on $\sim 43\%$ of instances; on this dataset, providing it as a CPLEX warm start did not measurably change solve times either way.

a) Cross-puzzle synthesis. The three puzzles span an interesting axis of constraint locality. Mosaic’s constraints are purely local (each binary appears in at most nine clue equations); CPLEX solves the LP relaxation at the root and the gap to integer-feasibility is essentially zero. Range adds a global *connectivity* requirement, which our two formulations make explicit either statically (flow with Big-M) or dynamically (lazy cuts); the latter wins by orders of magnitude on small instances and only converges to comparable performance on the larger ones. Galaxies adds a structural *symmetry* axis on top: cells gain a long-range constraint to their mirror images, but the partitioning is also strongly anchor-driven, which means the LP relaxation remains tight and only a small fraction of instances ever need the connectivity callback to fire. Across the three, the lesson is that ILP is at its best when constraints are local *or* when the global constraint can be lazily separated by a polynomial-time oracle (BFS in our case).

REFERENCES

- [1] S. Tatham, *Simon Tatham’s Portable Puzzle Collection*, <https://www.chiark.greenend.org.uk/~sgtatham/puzzles/>, accessed May 2026.
- [2] E. Friedman, *Spiral Galaxies Puzzles are NP-complete*, Stetson University, DeLand, FL.
- [3] G. Fertin, S. Jamshidi, and C. Komusiewicz, *Towards an Algorithmic Guide to Spiral Galaxies*, Université de Nantes, LINA – UMR CNRS 6241, France.